

Advanced Programming

NiceGUI Framework (2)

Rainer Telesko



Table of content

- Professional NiceGUI layout
 - Core components
 - Layout patterns
 - Tailwind CSS cheat sheet
- Sessions
- User storage
- MVC pattern
- Summary

Professional NiceGUI layout

For professional applications it is necessary to have a professional layout

- Common layout patterns: What do I want to design? (e.g. login page etc.)
- Components / containers: Define the main building blocks of a page (e.g. vertical stack of elements) and the layout of their children
- Look&Feel: How to optimize the layout? (e.g. coloring, fonts, margins, padding etc.)
→ NiceGUI components are based on Quasar (Vue), but you style and position them primarily using Tailwind utility classes via `.classes()`.

Tailwind

- Tailwind = utility-first CSS framework (<https://tailwindcss.com/>)
- “Utility-first” means you style things using small, single-purpose classes (utilities) instead of writing custom CSS
- In NiceGUI used via `.classes()`
- Goal = speed + consistency + no CSS overhead

Your original CSS

```
<> CSS

.button {
  padding: 1rem;
  background: blue;
  border-radius: 0.5rem;
}
```

Tailwind equivalent

```
<> Python

ui.button('Click me').classes('p-4 bg-blue-500 rounded-lg')
```

Tailwind classes for NiceGUI users

🎨 Tailwind CSS in NiceGUI – Quick Reference

📏 Layout & Sizing

Class	Meaning
w-full	Full width (100%)
w-1/2, w-1/3, w-1/4	Fraction of parent
w-screen	Full viewport width
h-full	Full height
h-screen	Full viewport height

📐 Spacing

Class	Meaning
p-4	Padding all sides
px-4	Horizontal padding
py-2	Vertical padding
m-4	Margin
mx-auto	Center horizontally

Cheatsheet available
on Moodle

Components / containers

- `ui.row()` — horizontal flex container
- `ui.column()` — vertical flex container
- `ui.card() + ui.card_section()` — grouped, padded “panel”
- `ui.grid()` — grid layout (best for dashboards and responsive “cards”)
- **Tabs:** `ui.tabs() + ui.tab_panels()` — switch between “pages” of content

`ui.row()` — horizontal flex container

- For placing elements next to each other.
Under the hood it behaves like a flex row

Things you'll do a lot with rows

- **Alignment:** `items-start|center|end` (vertical), `justify-start|center|between|end` (horizontal)
- **Spacing:** `gap-*`
- **Wrapping:** add `flex-wrap` if you want children to wrap on small screens
- **Full-width layouts:** `w-full`, sometimes `justify-between` and `flex-grow` on children

```
from nicegui import ui

with ui.row().classes('w-full items-center gap-3'):
    # w-full      -> row spans the available width
    # items-center -> vertical align children in the middle (cross-axis)
    # gap-3       -> spacing between children

    ui.icon('menu')
    ui.label('Title').classes('text-lg font-semibold')

    # A common pattern: push items to the far right
    ui.space() # Quasar spacer; expands to fill available space

    ui.button('Save')
    ui.button('Cancel').props('flat')
```

`ui.column()` — vertical flex container

- `ui.column()` stacks children top-to-bottom. Also flex-based, but direction is column.

Common column patterns

- “Form columns” using `gap-*` and fixed label width + growable input (`w-24 + flex-grow`)
- Centered content: `items-center`
- Full-page center: `min-h-screen items-center justify-center`

```
with ui.column().classes('w-full gap-4'):
    # gap-4 -> vertical spacing between items
    ui.label('Profile').classes('text-xl font-bold')

    with ui.row().classes('gap-3 items-center'):
        ui.label('Name').classes('w-24 text-gray-600')
        ui.input().classes('flex-grow') # flex-grow makes the input take remaining space

    with ui.row().classes('gap-3 items-center'):
        ui.label('Email').classes('w-24 text-gray-600')
        ui.input().classes('flex-grow')
```


`ui.card()` + `ui.card_section()` — grouped “panel”

- Use cards to visually group controls.
Cards are especially good as “modules” inside grids/columns.

Card tips

- Use multiple `card_section()` blocks for “header / body / footer”
- Add a “footer row” with `justify-end`
- Use `p-*` on card or sections (depending on how you want spacing)

```
with ui.card().classes('w-full max-w-lg p-4'):
    # p-4 -> padding inside the card
    # max-w-lg -> cap width so it doesn't stretch too far on big screens

    with ui.card_section():
        ui.label('Settings').classes('text-lg font-semibold')

    with ui.card_section().classes('gap-3'):
        # card_section is just a sub-container with nice spacing defaults
        ui.switch('Enable feature')
        ui.select(['Low', 'Medium', 'High'], label='Quality')

    with ui.card_section().classes('flex justify-end gap-2'):
        ui.button('Reset').props('flat')
        ui.button('Save')
```

`ui.grid()` — grid layout (best for dashboards / resp. “cards”)

- `ui.grid(columns=...)` is for multi-column layouts. It’s perfect for “N cards across”.

Grid tips

- Use responsive column definitions (`'1 sm:2 md:3 xl:4'`)
- Use `gap-*` for consistent spacing
- Inside each grid cell, use `column()` or `row()` to structure content

```
with ui.grid(columns='1 sm:2 lg:3').classes('w-full gap-4'):
    # 1 column on tiny screens
    # 2 columns from sm breakpoint upward
    # 3 columns from lg breakpoint upward
    for i in range(6):
        with ui.card().classes('p-4'):
            ui.label(f'Card {i}').classes('font-semibold')
            ui.label('Some description').classes('text-sm text-gray-600')
```

`ui.tabs()` / `ui.tab_panels()` — “pages” of content

- Tabs are for organizing multiple groups of controls without navigation routes.

Tab tips

- Put `p-*` on each panel for consistent padding
- For forms inside a tab, use `column().classes('gap-*')`

```
with ui.tabs().classes('w-full') as tabs:
    general = ui.tab('General')
    advanced = ui.tab('Advanced')
    about = ui.tab('About')

with ui.tab_panels(tabs, value=general).classes('w-full'):
    with ui.tab_panel(general).classes('p-4'):
        ui.input('Project name')
        ui.switch('Enable notifications')

    with ui.tab_panel(advanced).classes('p-4'):
        ui.input('API Key')
        ui.checkbox('Verbose logging')

    with ui.tab_panel(about).classes('p-4'):
        ui.label('Version 1.2.3')
```

Layout Pattern 1: Centered Form Page

- Goal
 - Center content vertically and horizontally
 - Constrain width
 - Clean spacing

Layout Strategy:
Column (full screen)
-> Card (max width)
-> Column (form fields)

Layout Pattern 2: Sidebar + Main Content

- Goal
 - Fixed sidebar
 - Flexible main area
 - Full height layout

Layout Strategy:
Row (full height)
-> Sidebar (fixed width)
-> Main (flex-grow)

Layout Pattern 3: Responsive Card Grid

- Goal
 - Responsive dashboard layout
 - Automatic column adjustment

Layout Strategy:

Column

- > Grid (responsive columns)
- > Cards

Layout Pattern 4: Toolbar + Content

- Goal
 - Top toolbar
 - Title left, actions right
 - Content below

Layout Strategy:
Column
-> Row (toolbar)
-> Content Area

Layout Pattern 5: Split View

- Goal
 - Master-detail layout
 - Resizable panels

Layout Strategy:

Splitter

-> Left Panel

-> Right Panel

Layout Pattern 6: Sectioned Page

- Goal
 - Break long content into sections
 - Clear visual grouping

Layout Strategy:
Column (max width)
-> Card (Section 1)
-> Card (Section 2)

Layout Pattern 7: Full App Shell

- Goal
 - Header
 - Sidebar
 - Main content
 - Full height structure

Layout Strategy:
Column (full height)
-> Header
-> Row
 -> Sidebar
 -> Main

NiceGUI Layout Patterns – Business Use Overview

#	Pattern	Typical Use Cases	Choose When...	
1	Centered Form	Login, Signup, Reset, Wizard step	Single focused task	
2	Sidebar + Main	Admin, CRM, ERP, Backoffice	Persistent navigation needed	
3	Card Grid	KPIs, Dashboards, Catalogs	Showing multiple summary blocks	
4	Toolbar + Content	CRUD pages, Tables, Search	Actions + structured data	
5	Split View	Tickets, Email, File manager	List + detail editing	
6	Sectioned Page	Settings, Profile, Config	Long grouped forms	
7	Full App Shell	SaaS platforms, Enterprise apps	Large scalable systems	

Sessions

- In NiceGUI, a session represents one connected user and their isolated UI and state, from the moment they open the app until they disconnect
- Each user gets their *own session* (a unique session/client ID tied to the WebSocket connection)
- ID is not tied to a real person or identity
- A NiceGUI session typically includes:
 - UI tree (all ui.* elements created for that user)
 - Callback bindings (event handlers)
 - Per-user state (variables created in that session)
 - WebSocket connection to the server
- This means
 - Two users never share UI elements
 - One user refreshing the page creates a new session

Sessions

- A session is created when:
 - A user opens a NiceGUI page (`@ui.page`)
 - Or connects to the default / page
- Each browser tab → one session
- “Ending” a session
 - Disconnect the current session, e.g. logout
 - Redirect to new page
 - Clear session data (but keep session alive)
 - Auto-end session on disconnect (default behavior), e.g. when the browser tab is closed

User Storage – the idea

- Idea: “Remember the user across pages & tabs”
- `app.storage.user` is a persistent key-value storage that belongs to one user (browser), not just one session

```
app.storage.user.update({  
  'authenticated': True,  
  'username': username.value,  
  'role': user['role']  
})
```

👉 This means:

-  The user is logged in
-  Who they are (`username`)
-  What they can do (`role`)

Feature 	<code>app.storage.user</code>
Storage Location	Server (stored as a JSON file by default)
Across Browser Tabs	Yes (data is shared between all tabs of the same user)
Across Browsers	No (data is unique to the specific browser/user session)
Server Restarts	Yes (persists because it is written to the server's disk)
Page Reloads	Yes (data remains available after refreshing the page)
Security	Higher (content is kept on the server; only a session ID is in the cookie)
Data Requirements	Must be JSON-serializable
Setup Required	Needs a <code>storage_secret</code> passed in <code>ui.run()</code>

User Storage – the workflow (see login.py)

- Step 1: Login (Authentication):

We store identity after login

```
if user and user['password'] == password.value:  
    app.storage.user.update({  
        'authenticated': True,  
        'username': username.value,  
        'role': user['role']  
    })
```

- Step 2: Protect pages (Authorization):

Only logged-in users can enter

```
if not user_data.get('authenticated', False):  
    ui.navigate.to('/login')
```

- Step 3: Role-based access:

Even logged-in users may be restricted

```
if user_data.get('role') != 'admin':  
    ui.label('Access Denied: Admins Only')
```

- Step 4: Logout:

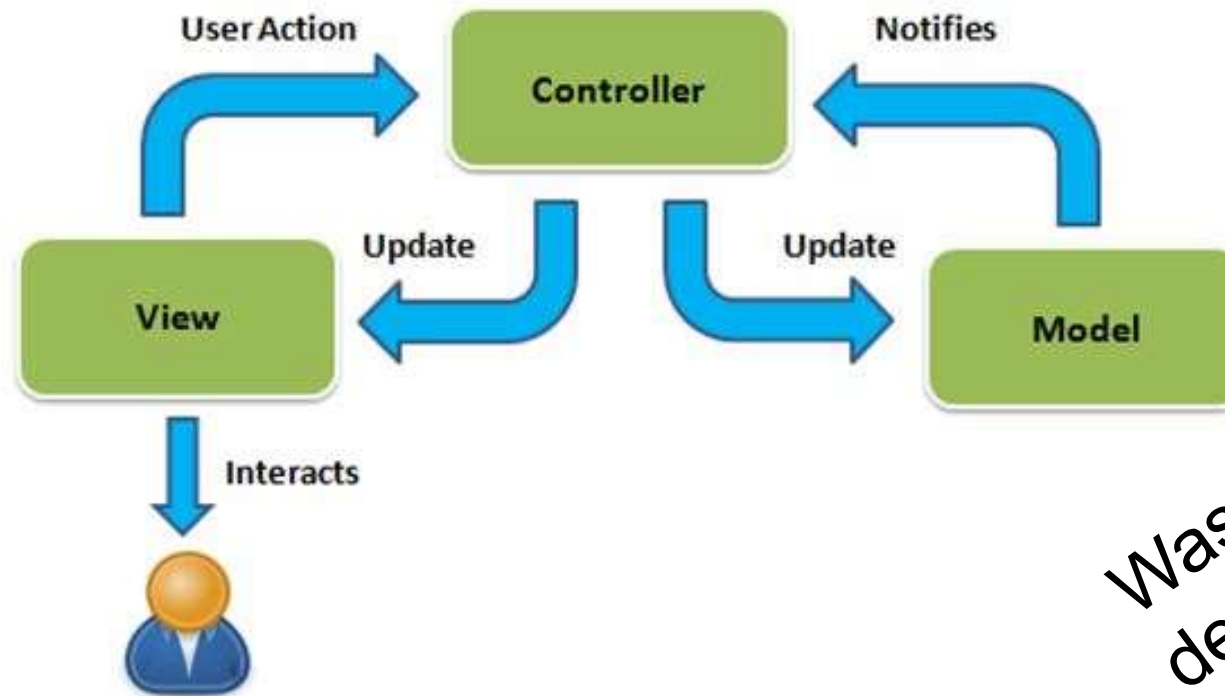
Removing login state, username and role

```
app.storage.user.clear()
```

Introducing MVC

- Model–View–Controller architectural pattern
- Separates responsibilities
- Improves maintainability
- MVC in Simple Terms
- Model: data
- View: UI
- Controller: logic

Introducing MVC



Was explained in the
design pattern lecture

Source: <https://thedotnetguide.com/mvc-design-pattern/>, access: 2026-01-16

Why MVC Fits NiceGUI

- Callbacks act as controllers
 - They receive user events
 - Call model methods
 - Update the view
- UI elements are views
 - Displays state
 - Emits events
 - Do not contain business logic
- Models are pure Python
 - Plain Python objects
 - Dataclasses
 - Pydantic models
 - Database-backed models

Pizzeria Reference app is not true MVC ...



- The best label for it is: layered architecture with controllers + services + DAOs, and the NiceGUI part acts as the UI layer
- Components
 - Pages (UI) → View-like
 - Controllers → Controller 
 - Services + DAOs + DB → split “Model” 
 - The “Model” is fragmented across layers

Summary



- Professional NiceGUI apps require structured layout patterns, clear components, and consistent styling.
- Core layout containers include: `ui.row`, `ui.column`, `ui.card`, `ui.grid` and `ui.tabs`.
- Seven common layout patterns cover typical business scenarios.
- Styling and positioning are primarily done using Tailwind CSS utility classes via `.classes()`.
- Sessions isolate users
- The MVC pattern improves structure and maintainability:
 - Model = data
 - View = UI elements
 - Controller = logic (callbacks)
- In NiceGUI, sessions isolate users by giving each browser tab its own independent UI and state, which is automatically created on connection and ends on logout or disconnect.

Exercises



- Create a professional layout for the screens using layout patterns
- Implement the selected screens with NiceGUI (see the Pizzeria reference project for a fully-fledged OO-based implementation)